Data Preprocessing

KJR

Content

- Clustering & Classification
- Difference Between Scalar, Vector, Matrix and Tensor
- Data Cleaning & Preprocessing in Python
- Feature Extraction Techniques
- Feature Selection Techniques

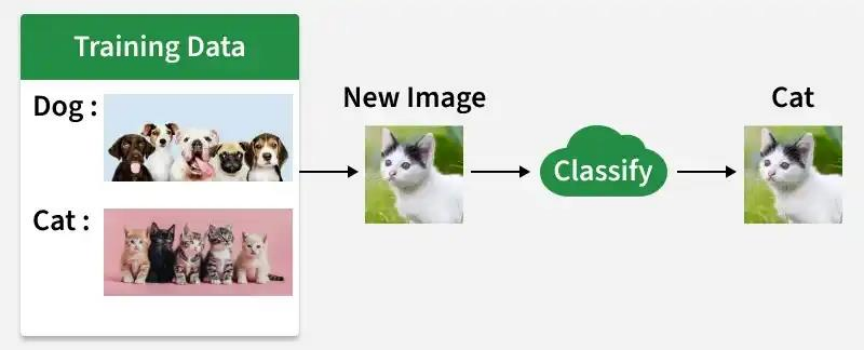
➢ Classification

Classification is a supervised machine learning technique used to predict labels or categories from input data. It assigns each data point to a predefined class based on learned patterns.

•**Predict categories**: Determines the class of new data points.

•**Uses labeled data:** Trained on datasets where the correct class is known.

•**Common examples**: Spam vs non spam emails, diseased vs. healthy patients.



➢ Clustering in Machine Learning

Clustering is an unsupervised machine learning technique used to group similar data points together without using labelled data. It helps discover hidden patterns or natural groupings in datasets by placing similar data points into the same cluster.

•Discover the natural grouping or structure in unlabeled data without predefined categories.

•Data points are assigned to clusters based on similarity or distance measures.

•Uses Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.

Comparison between Classification and Clustering:

Parameter	CLASSIFICATION	CLUSTERING
Type	used for supervised learning	used for unsupervised learning
Basic	process of classifying the input instances based on their corresponding class labels	grouping the instances based on their similarity without the help of class labels
Need	it has labels so there is need of training and testing dataset for verifying the model created	there is no need of training and testing dataset
Complexity	more complex as compared to clustering	less complex as compared to classification
Example Algorithms	Logistic regression, Naive Bayes classifier, Support vector machines, etc.	k-means clustering algorithm, Fuzzy c-means clustering algorithm, Gaussian (EM) clustering algorithm, etc.

Difference Between Scalar, Vector, Matrix and Tensor

Example-1: Let's see a python example to understand how a scalar is represented.

```
scalar = 8.4
print(scalar)
```

Output

```
8.4
```

Example-2: Let's see a python example to understand how vectors are represented.

```
import numpy as np
vector = np.array([2, -3, 1.5])
print(vector)
```

Output

```
[ 2.  -3.   1.5]
```

Example-3: Let's see a python example to understand how matrix are represented.

```
import numpy as np
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print(matrix)
```

Output

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Example-4: Let's see a python example to understand how tensor are represented.

```
import numpy as np
tensor = np.array([[[1, 2], [3, 4]],
                   [[5, 6], [7, 8]]])
print(tensor)
```

Output

```
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Aspect	Scalar	Vector	Matrix	Tensor
Dimensionality	0D	1D	2D	≥3D
Representation	Single numerical value	Ordered array of values	Two-dimensional grid of values	Multi-dimensional array of values
Usage	Represent basic quantities	Represent features or directions	Organize data in tabular form	Represent complex data structures
Examples	Error metric, probability	Feature vector, gradient	Data matrix, weight matrix	Image tensor, video tensor
Manipulation	Arithmetic operations	Linear algebra operations	Matrix operations, transformations	Tensor operations in deep learning
Data Representation	Point in space	Direction and magnitude	Rows and columns	Multi-dimensional relationships
Applications	Basic calculations, statistics	ML features, embeddings	Data manipulation, analysis	Deep learning, NLP, computer vision

❑Data Cleaning & Preprocessing

Data cleaning means fixing bad data in your data set.

Bad data could be:

- ✓ Empty cells
- ✓ Data in wrong format
- ✓ Wrong data
- ✓ Duplicates

Data cleaning involves identifying and removing any missing, duplicate or irrelevant data.

•Raw data (log file, transactions, audio /video recordings, etc) is often noisy, incomplete and inconsistent which can negatively impact the accuracy of the model.
•The goal of data cleaning is to ensure that the data is accurate, consistent and free of errors.
•Clean datasets are also important in EDA (Exploratory Data Analysis), which enhances the interpretability of data so that the right actions can be taken based on insights.

➢ Cleaning Empty Cells

1. Remove Rows

One way to deal with empty cells is to remove rows that contain empty cells. This is usually OK, since data sets can be very big, and removing a few rows will not have a big impact on the result.

Program – 1:

```
import pandas as pd
df = pd.read_csv('data.csv')
new_df = df.dropna()
print(new_df.to_string())
```

Return a **new Data Frame** with no empty cells.

Note: By default, the `dropna()` method returns a *new* DataFrame, and will not change the original.

Program – 2:

If you want to change the original DataFrame, use the `inplace = True` argument:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.dropna(inplace = True)
print(df.to_string())
```

Remove all rows with NULL values.

Note: Now, the `dropna(inplace = True)` will NOT return a new DataFrame, but it will remove all rows containing NULL values from the original DataFrame.

2. Replace Empty Values

Another way of dealing with empty cells is to insert a ***new* value** instead.
This way you do not have to delete entire rows just because of some empty cells.
The `fillna()` method allows us to replace empty cells with a value:

Program – 3:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.fillna(130, inplace = True)
```

Replace NULL values with the number 130

3. Replace Only For Specified Columns

`Program – 3` replaces all empty cells in the whole Data Frame.
To only replace empty values for one column, specify the ***column name*** for the DataFrame:

Program – 4:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.fillna({"Calories": 130}, inplace=True)
```

Replace NULL values in the "Calories" columns with the number 130

4. Replace Using Mean, Median, or Mode

A common way to replace empty cells, is to calculate the mean, median or mode value of the column.
Pandas uses the mean() median() and mode() methods to calculate the respective values for a specified column:
Calculate the MEAN, and replace any empty values with it:

Program – 5:

```
import pandas as pd
df = pd.read_csv('data.csv')
x = df["Calories"].mean()
df.fillna({"Calories": x}, inplace=True)
```

Mean = the average value (the sum of all values divided by number of values).

Program – 6:

```
import pandas as pd
df = pd.read_csv('data.csv')
x = df["Calories"].median()
df.fillna({"Calories": x}, inplace=True)
```

Median = the value in the middle, after you have sorted all values ascending.

Program – 7:

```
import pandas as pd
df = pd.read_csv('data.csv')
x = df["Calories"].mode()[0]
df.fillna({"Calories": x}, inplace=True)
```

Mode = the value that appears most frequently.

Cleaning Data of Wrong Format

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	20201226	100	120	250.0

```
import pandas as pd
df = pd.read_csv('data.csv')
df['Date'] = pd.to_datetime(df['Date'], format='mixed')
print(df.to_string())
```

22	45	NaT	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	'2020/12/26'	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	NaN
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

As you can see from the result, the date in row 26 was fixed, but the empty date in row 22 got a NaT (Not a Time) value, in other words an empty value. One way to deal with empty values is simply removing the entire row.

Removing Rows

The result from the converting in the example above gave us a NaT value, which can be handled as a NULL value, and we can remove the row by using the dropna() method.

Remove rows with a NULL value in the "Date" column:

```
df.dropna(subset=['Date'], inplace = True)
```

➢ Removing Duplicates

Duplicate rows are rows that have been registered more than one time.

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.duplicated())
```

Returns `True` for every row that is a duplicate, otherwise `False`:

```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9     False
10    False
11    False
12     True
13    False
14    False
```

Removing Duplicates: To remove duplicates, use the `drop_duplicates()` method.

```
df.drop_duplicates(inplace = True)
```

The `(inplace = True)` will make sure that the method does NOT return a *new* DataFrame, but it will remove all duplicates from the *original* DataFrame.

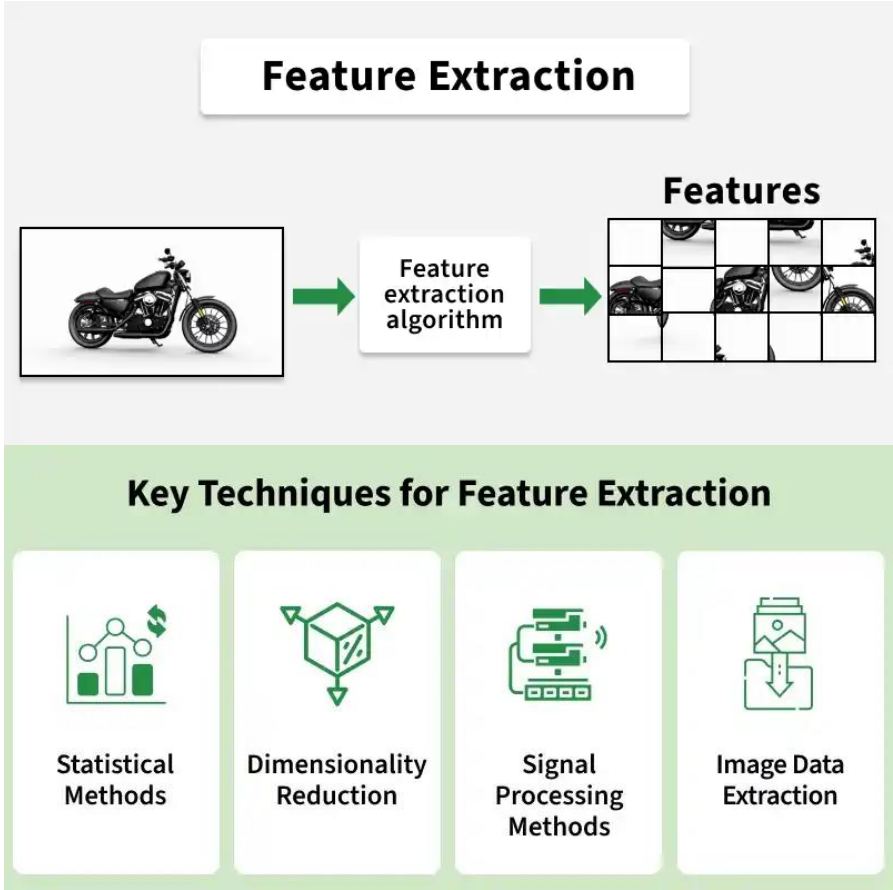
❑ What is Feature Extraction?

Feature extraction transforms raw data into meaningful and structured features that machine learning models can easily interpret. It organizes complex data into clear and useful variables so that patterns and relationships in the data can be understood more easily. This step prepares the data in a form that supports effective analysis and prediction.

•Converts raw and unstructured data into useful features

•Represents the important characteristics of the dataset through clear variables

•Helps machine learning models learn patterns and relationships in data by providing meaningful inputs



❑ Key Techniques for Feature Extraction

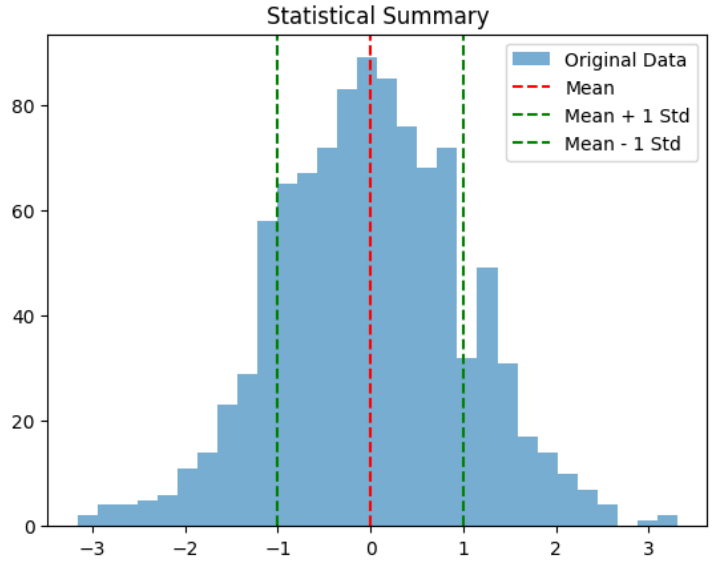
There are various techniques for extracting meaningful features from different types of data:

1. Statistical
2. Dimensionality reduction
3. Feature extract for text
4. Signal processing
5. Image data extract

1. Statistical Methods

Statistical methods are used in feature extraction to summarize and explain patterns of data. Common data attributes include:

•**Mean**: The average value of a dataset.
•**Median**: The middle value when it is sorted in ascending order.
•**Standard Deviation**: A measure of the spread or dispersion of a sample.
•**Correlation and Covariance**: Measures of the linear relationship between two or more factors.
•**Regression Analysis**: A way to model the link between a dependent variable and one or more independent factors.



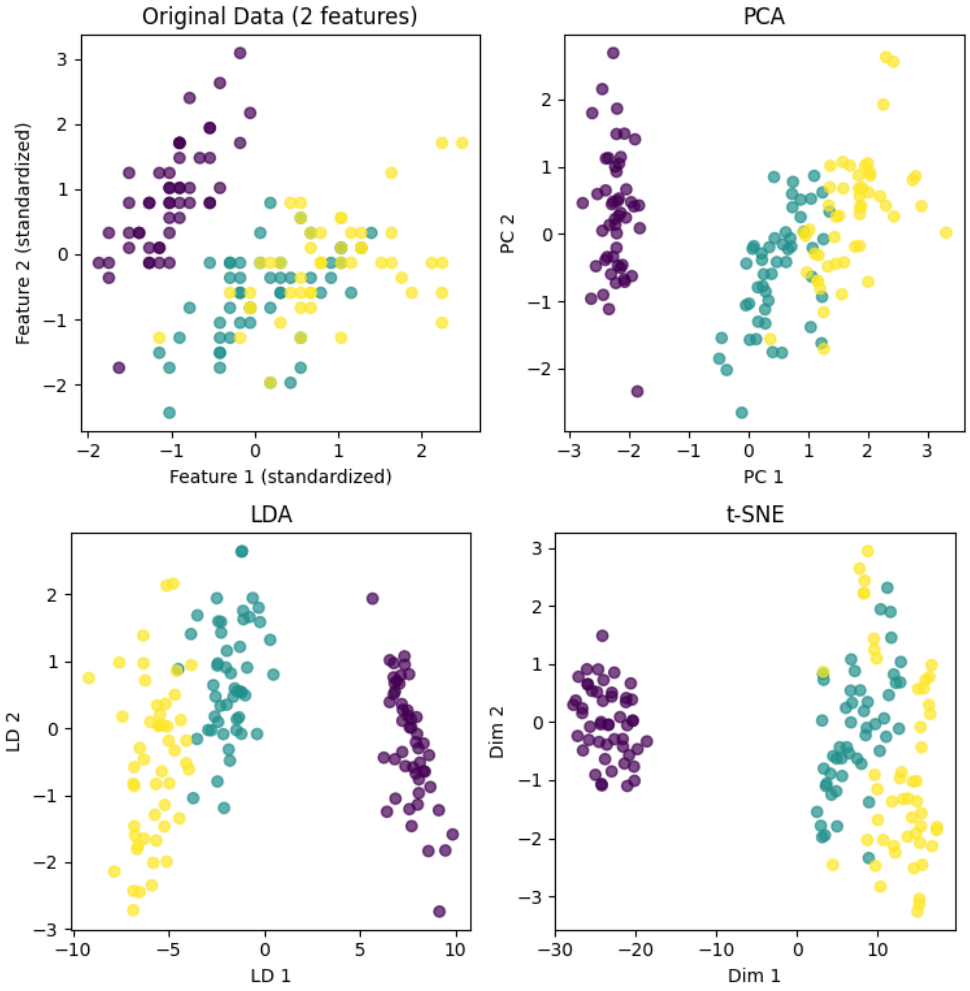
2. Dimensionality Reduction

Dimensionality reduction reduces the number of features without losing important information. Some popular methods are:

- **Principal Component Analysis:** It selects variables that account for most of the data's variation, simplifying the dataset by focusing on the most important components.

- **Linear Discriminant Analysis (LDA):** It finds the best combination of features to separate different classes, maximizing class separability for better classification.

- **t-Distributed Stochastic Neighbor Embedding (t-SNE)**: A technique that reduces high-dimensional data into two or three dimensions ideal for visualizing complex datasets.



3. Feature Extraction for Textual Data

In Natural Language Processing (NLP), we often convert raw text into a format that machine learning models can understand.

1.Bag of Words (BoW): Represents a document by counting word frequencies, ignoring word order, useful for basic text classification.

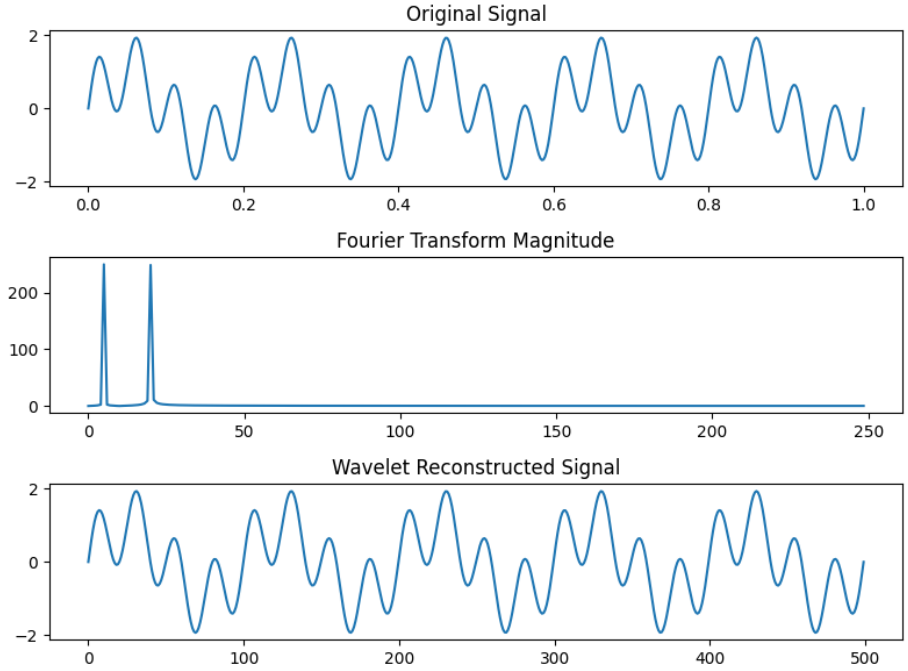
2.Term Frequency-Inverse Document Frequency (TF-IDF): Adjusts word importance based on frequency in a specific document compared to all documents, highlighting unique terms.

4. Signal Processing Methods

It is used for analyzing time-series, audio and sensor data:

1.Fourier Transform: It converts a signal from the time domain to the frequency domain to analyze its frequency components.

2.Wavelet Transform: It analyzes signals that vary over time, offering both time and frequency information for non-stationary signals.

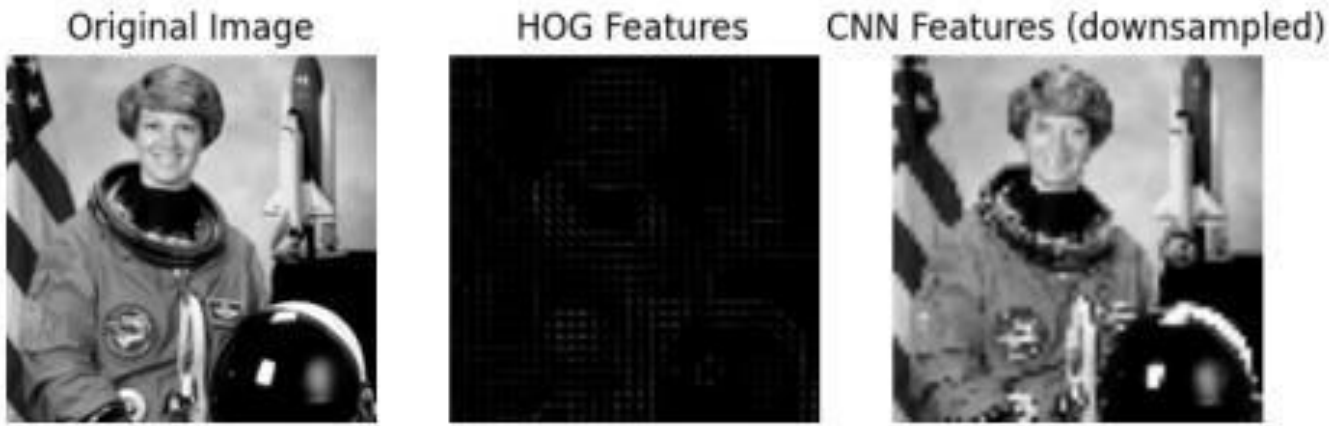


5. Image Data Extraction

Techniques for extracting features from images:

1.Histogram of Oriented Gradients (HOG): This technique finds the distribution of intensity gradients or edge directions in an image. It's used in object detection and recognition tasks.

2.Convolutional Neural Networks (CNN) Features: They learn hierarchical features from images through layers of convolutions, ideal for classification and detection tasks.



❑ Feature extraction has various Challenges

- **Managing High-Dimensional Data:** Extracting relevant features from large, complex datasets can be difficult.
- **Risk of Overfitting or Underfitting:** Too many or too few features can hurt model accuracy and generalization.
- **Computational Costs:** Complex methods may require heavy resources, limiting use with big or real-time data.
- **Redundant or Irrelevant Features:** Overlapping or noisy features can confuse models and reduce efficiency.

❑ Feature Selection Techniques in Machine Learning

Feature selection is the process of choosing only the most useful input features for a machine learning model. It helps improve model performance, reduces noise and makes results easier to understand.

- ✓ Helps remove irrelevant and redundant features
- ✓ Improves accuracy and reduces overfitting
- ✓ Speeds up model training
- ✓ Makes models simpler and easier to interpret

➢ Need of Feature Selection

Feature selection methods are essential in data science and machine learning for several key reasons:

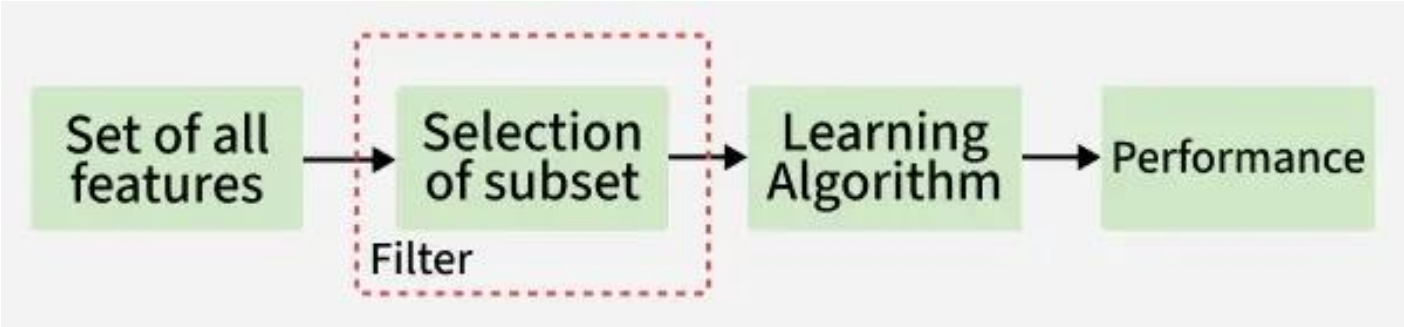
- **Improved Accuracy**: Models learn better when trained on only important features.
- **Faster Training**: Fewer features reduce computation time.
- **Greater Interpretability**: With fewer inputs, understanding model behavior becomes easier.
- **Avoiding the Curse of Dimensionality**: Reduces complexity when working with high-dimensional data.

Types:

1. Filter
2. Wrapper
3. Embedded

1. Filter Methods

Filter methods evaluate each feature independently with target variable. Feature with high correlation with target variable are selected as it means this feature has some relation and can help us in making predictions. These methods are used in the preprocessing phase to remove irrelevant or redundant features based on statistical tests (correlation) or other criteria.



Common Filter Techniques

- **Information Gain:** Measures reduction in entropy when a feature is used.
- **Chi-square test:** Checks the relationship between categorical features.
- **Fisher's Score:** Ranks features based on class separability.
- **Pearson's Correlation Coefficient:** Measures linear relationship between two continuous variables.
- **Variance Threshold:** Removes features with very low variance.
- **Mean Absolute Difference:** Similar to variance threshold but uses absolute differences.
- **Dispersion ratio:** Ratio of arithmetic mean to geometric mean; higher values indicate useful features.

2. Wrapper methods

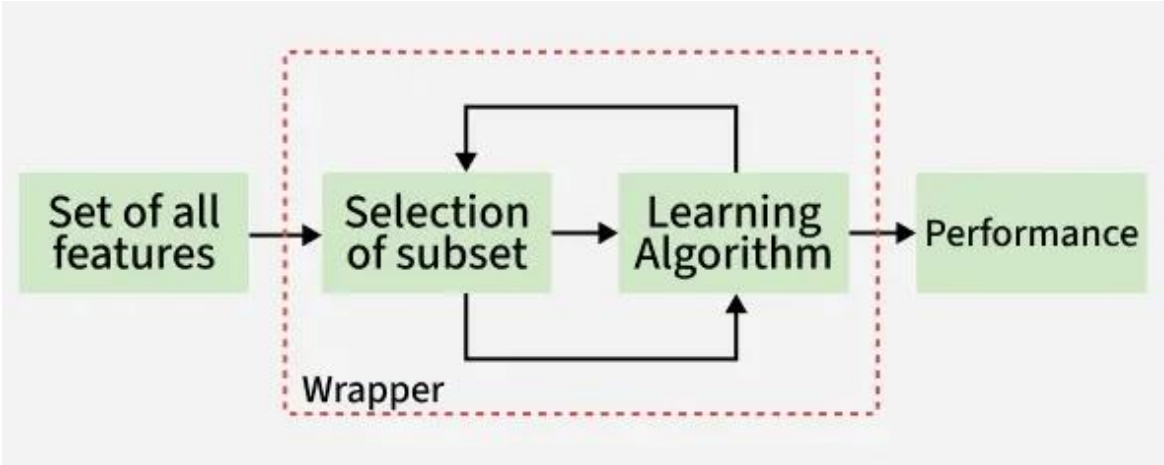
Wrapper methods are also referred as greedy algorithms that train algorithm. They use different combination of features and compute relation between these subset features and target variable and based on conclusion addition and removal of features are done. Stopping criteria for selecting the best subset are usually pre-defined by the person training the model such as when the performance of the model decreases or a specific number of features are achieved.

Common Wrapper Techniques

•**Forward Selection:** Start with no features and add one at a time based on improvement.

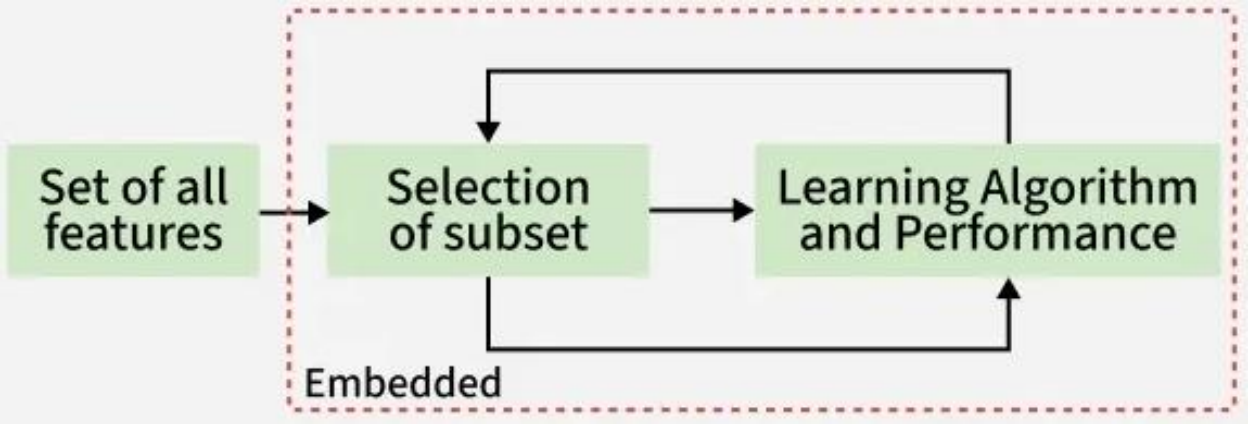
•**Backward Elimination:** Start with all features and remove the least useful ones.

•**Recursive Feature Elimination (RFE):** Removes the least important features step by step.



3. Embedded methods

Embedded methods perform feature selection during the model training process. They combine the benefits of both filter and wrapper methods. Feature selection is integrated into the model training allowing the model to select the most relevant features based on the training process dynamically.



Common Embedded Techniques

- **L1 Regularization (Lasso):** Keeps only features with non-zero coefficients.
- **Decision Trees** and **Random Forests:** Select features based on impurity reduction.
- **Gradient Boosting:** Pick features that reduce prediction error the most

❑ Feature Selection vs. Feature Extraction

Since Feature Selection and Feature Extraction are related but not the same, let's quickly see the key differences between them for a better understanding:

Aspect	Feature Selection	Feature Extraction
Definition	Selecting a subset of relevant features from the original set	Transforming the original features into a new set of features
Purpose	Reduce dimensionality	Transform data into a more manageable or informative representation
Process	Filtering, wrapper methods, embedded methods	Signal processing, statistical techniques, transformation algorithms
Output	Subset of selected features	New set of transformed features
Computational Cost	Lower cost	May be higher, especially for complex transformations
Interpretability	Retains interpretability of original features	May lose interpretability depending on transformation

See you in the next lecture…